

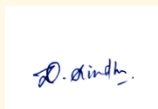
SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Debugging Test

Course Code:	DSA0502	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 3 – Debugging Test
Weightage:		Total Marks:	25 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	D. Sindhu	Reg. No.:	192424197
Section / Batch:	2024	Date of Submission:	02.08.2026
Faculty In-charge:	Dr. B.SHYAMALA GOWRI	Project Mode:	Individual Submission

Code of Conduct

I D. Sindhu (192424197) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature: _____

Error Identification(5)	Code Correction(8)	SQL Query Accuracy(5)	Explanation and Justification(4)	Program Execution and Output(3)	Total (25 Marks)

Q1. Debug the SQLite Program The following program is intended to create a table and insert student records. However, it contains multiple errors.

```
import sqlite3
conn = sqlite3.connect(college.db)
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE Student(
id INTEGER PRIMARY KEY
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1,'John',90)")
conn.close()
```

Task:

1. Identify **at least five errors**.
 2. Rewrite the corrected program.
 3. Explain the purpose of each correction.
-

Q2. Debug the CRUD Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchone():
    print(row)
cursor.execute("UPDATE Student SET marks=95")
cursor.execute("DELETE FROM Student")
conn.close()
```

Task:

1. Identify all logical and syntax errors.
 2. Explain why every record is modified or deleted.
 3. Rewrite the corrected program.
-

Q3. Debug the MySQL Connection

```
import mysql.connector
connection=mysql.connector.connect(host="localhost",user="root"password="root",database="college"
)
cursor=connection.cursor()
cursor.execute("SELECT * FROM student")
for i in cursor:print(i)
```

Task

- Identify every syntax and indentation error.

- Rewrite the corrected program.
- Explain why the original program fails.

Q4. Debug the PostgreSQL Program

```
import psycopg2
conn=psycopg2.connect(dbname="college",user="postgres",password="admin")
)
cursor=conn.cursor()
cursor.execute("INSERT INTO Student VALUES(%s,%s,%s)",(1,"Ram"))
conn.commit()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())
conn.close()
```

Task

- Identify all errors.
- Rewrite the corrected program.
- Explain the effect of parameter mismatch.

Q5. Multiple Errors in SQL Statements

```
cursor.execute("CREATE TABLE Student(id INTEGER,name TEXT marks INTEGER)")
cursor.execute("INSERT Student VALUES(1,'Anu',89)")
cursor.execute("SELECT name marks FROM Student")
cursor.execute("UPDATE Student marks=90 WHERE id=1")
cursor.execute("DELETE Student WHERE id=2")
```

Task

1. Identify every SQL error.
2. Write the corrected SQL statements.
3. State which SQL command (DDL/DML/DQL) each statement belongs to.

Q6. Debug the Transaction Program

```
import sqlite3
conn=sqlite3.connect("college.db")
cursor=conn.cursor()
cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
cursor.execute("INSERT INTO Student VALUES(2,'John',80)")
conn.rollback()
conn.commit()
conn.close()
```

Task

- Identify logical errors related to transactions.
- Explain the effect of rollback() and commit().
- Rewrite the corrected program.

Q7. Debug the Parameterized Query id=2
cursor.execute("SELECT * FROM Student WHERE id="+id)
print(cursor.fetchall())

Task

1. Identify all errors.
2. Explain why the code is unsafe.
3. Rewrite the query using parameterized SQL.

Q8. Debug the Database Fetch Program cursor.execute("SELECT * FROM Student")
rows=cursor.fetchall()
print(rows[20])
cursor.close()
cursor.execute("SELECT * FROM Student")

Task

- Identify runtime errors.
- Explain why the program fails.
- Rewrite the corrected program.

Q9. Integrated Debugging Problem import sqlite3
conn=sqlite3.connect("college.db")
cursor=conn.cursor()
cursor.execute("CREATE TABLE Student(id INTEGER PRIMARY KEY,name TEXT,marks INTEGER)")
cursor.execute("INSERT INTO Student VALUES(1,'Asha',90)")
cursor.execute("INSERT INTO Student VALUES(1,'Raj',85)")
cursor.execute("UPDATE Student SET mark=95 WHERE id=1")
cursor.execute("SELECT name,marks FROM Students")
print(cursor.fetchall())
conn.close()

Task

1. Identify **at least six errors**.
2. Rewrite the corrected program.
3. Explain the exception generated by each error.

Q10. Complete Database Application Debugging The following application should perform all CRUD operations but produces incorrect output.

import sqlite3

```
conn=sqlite3.connect("college.db")
cursor=conn.cursor()
cursor.execute("INSERT INTO Student VALUES(3,'Kiran',90)")
cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())
cursor.execute("UPDATE Student SET marks=95")
cursor.execute("DELETE FROM Student WHERE id==3")
conn.close()
```

Task

1. Identify every syntax, logical, and runtime error.
2. Rewrite the complete corrected program.
3. Explain how each correction improves the program.
4. Predict the output after successful execution.

Q1. Debug the SQLite Program

Errors

1. college.db should be inside quotes.
2. conn.cursor should be conn.cursor().
3. Missing comma after id INTEGER PRIMARY KEY.
4. Missing comma after name TEXT.
5. conn.commit() is missing before closing.

Correct Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE Student(
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1,'John',90)")
conn.commit()
conn.close()
```

Purpose of Corrections

- Quotes → Database filename must be a string.
 - cursor() → Creates a cursor object.
 - Commas → Separate table columns correctly.
 - commit() → Saves changes permanently.
-

Q2. Debug the CRUD Program

Errors

1. Two statements written on one line.
2. fetchone() returns one row, not multiple rows.
3. Loop should use fetchall().
4. UPDATE Student SET marks=95 updates every record.
5. DELETE FROM Student deletes every record.
6. Missing commit().

Correct Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchall():
    print(row)
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
cursor.execute("DELETE FROM Student WHERE id=2")
conn.commit()
conn.close()
```

Why all records are modified/deleted

- No WHERE clause in UPDATE → every row updated.
 - No WHERE clause in DELETE → every row deleted.
-

Q3. Debug the MySQL Connection

Errors

1. Missing comma after user="root".
2. Poor indentation.
3. for loop should be properly indented.

Correct Program

```
import mysql.connector
connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="college"
)
cursor = connection.cursor()
cursor.execute("SELECT * FROM student")
for i in cursor:
    print(i)
connection.close()
```

Why original program fails

- Missing comma causes **SyntaxError**.

- Incorrect formatting makes code difficult to execute.
-

Q4. Debug the PostgreSQL Program

Errors

1. INSERT has three placeholders.
2. Only two values supplied.
3. Cursor not closed.

Correct Program

```
import psycopg2
conn = psycopg2.connect(
    dbname="college",
    user="postgres",
    password="admin"
)
cursor = conn.cursor()
cursor.execute(
    "INSERT INTO Student VALUES(%s,%s,%s)",
    (1, "Ram", 90)
)
conn.commit()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())
cursor.close()
conn.close()
```

Effect of Parameter Mismatch

- SQL expects **3 values**.
 - Only **2 values** supplied.
 - Raises **ProgrammingError**.
-

Q5. Multiple SQL Errors

Correct SQL Statements

```
CREATE TABLE Student(
id INTEGER,
name TEXT,
marks INTEGER
);
```

Type: DDL

```
INSERT INTO Student VALUES(1,'Anu',89);
```

Type: DML

```
SELECT name, marks FROM Student;
```

Type: DQL

```
UPDATE Student
SET marks=90
```

WHERE id=1;

Type: DML

DELETE FROM Student

WHERE id=2;

Type: DML

Errors

- Missing comma after TEXT
 - Missing INTO
 - Missing comma in SELECT
 - Missing SET
 - Missing FROM
-

Q6. Debug the Transaction Program

Logical Errors

1. Update occurs before insert.
2. Insert duplicates ID 2.
3. rollback() comes before commit().
4. Rollback cancels previous changes.

Correct Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
try:
    cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
    conn.commit()
except:
    conn.rollback()
finally:
    conn.close()
```

Effect

- commit() → Saves changes.
 - rollback() → Undoes uncommitted changes.
-

Q7. Debug the Parameterized Query

Errors

1. Cannot concatenate integer with string.
2. SQL Injection risk.

Correct Program

```
id = 2
cursor.execute(
    "SELECT * FROM Student WHERE id=?",
    (id,)
)
```



```
print(cursor.fetchall())
```

Why unsafe

Using string concatenation allows malicious SQL injection.

Q8. Debug the Database Fetch Program

Errors

1. rows[20] may not exist.
2. Cursor is closed.
3. Query executed after closing cursor.

Correct Program

```
cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()
if len(rows) > 20:
    print(rows[20])
else:
    print("Less than 21 records.")
cursor.close()
```

Why it fails

- IndexError if fewer than 21 rows.
 - ProgrammingError after cursor is closed.
-

Q9. Integrated Debugging Problem

Errors

1. Table already exists.
2. Duplicate primary key.
3. Wrong column name mark.
4. Wrong table name Students.
5. Missing commit().
6. Cursor not closed.

Correct Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
id INTEGER PRIMARY KEY,
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES(1,'Asha',90)")
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
cursor.execute("SELECT name, marks FROM Student")
```

```
print(cursor.fetchall())
conn.commit()
cursor.close()
conn.close()
```

Exceptions

- Duplicate ID → IntegrityError
 - Wrong column → OperationalError
 - Wrong table → OperationalError
-

Q10. Complete Database Application Debugging

Errors

1. Missing commit().
2. UPDATE changes every record.
3. == used in SQL (= preferred).
4. Fetching only one record.
5. No final verification after update/delete.

Correct Program

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("INSERT INTO Student VALUES(3,'Kiran',90)")
conn.commit()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchall())
cursor.execute(
    "UPDATE Student SET marks=95 WHERE id=3"
)
cursor.execute(
    "DELETE FROM Student WHERE id=3"
)
conn.commit()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchall())
cursor.close()
conn.close()
```

Improvements

- commit() saves changes.
- WHERE prevents updating all rows.
- = is standard SQL comparison.
- fetchall() displays complete table.
- Final SELECT verifies CRUD operations.

Expected Output (Example)

After Insert:

[(1, 'John', 90), (3, 'Kiran', 90)]

After Update and Delete:

[(1, 'John', 90)]
